

```
# Data Handling
import pandas as pd
import numpy as np
```

```
#Visualization
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report,
    mean_absolute_error,
    mean_squared_error,
    r2_score)
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)
```

```
# Classification Models
from sklearn.ensemble import RandomForestClassifier
# Regression Models
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
# Warnings
import warnings
warnings.filterwarnings('ignore')
# Display settings
pd.set_option('display.max_columns', None)
# Style
sns.set(style='whitegrid')
```

```
#Part A Importing and cleaning
#Importing dataset
df = pd.read_csv('/content/drive/MyDrive/Group_Project.csv')
```

```
# ENCODE CATEGORICAL VARIABLES
encoder = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = encoder.fit_transform(df[col])
print(df.head())
```

	Order_ID	Region	Product	Demand	Inventory	Lead_Time	Transport_Cost	\
0	1	3	1	122	332	5	1916	
1	2	1	0	110	518	7	3603	
2	3	2	0	220	441	12	3828	
3	4	0	2	159	509	6	360	
4	5	3	2	59	364	4	211	

	Warehouse_Capacity	Warehouse_Utilization	On_Time_Delivery
0	798	63	1
1	983	49	1
2	1220	77	0
3	808	66	1
4	1967	91	1

```
# Preview Dataset
print("\nFIRST 5 ROWS")
print(df.head())
print("\nDATASET INFO")
print(df.info())
print("\nMISSING VALUES")
print(df.isnull().sum())
print("\nSTATISTICAL SUMMARY")
print(df.describe())
```

FIRST 5 ROWS

	Order_ID	Region	Product	Demand	Inventory	Lead_Time	Transport_Cost	\
0	1	3	1	122	332	5	1916	
1	2	1	0	110	518	7	3603	
2	3	2	0	220	441	12	3828	
3	4	0	2	159	509	6	360	
4	5	3	2	59	364	4	211	

	Warehouse_Capacity	Warehouse_Utilization	On_Time_Delivery
0	798	63	1
1	983	49	1
2	1220	77	0
3	808	66	1
4	1967	91	1

DATASET INFO

<class 'pandas.core.frame.DataFrame'

RangeIndex: 5000 entries, 0 to 4999

Data columns (total 10 columns):

#	Column	Non-Null Count	Dtype
---	-----	-----	-----
0	Order_ID	5000 non-null	int64
1	Region	5000 non-null	int64
2	Product	5000 non-null	int64
3	Demand	5000 non-null	int64
4	Inventory	5000 non-null	int64
5	Lead_Time	5000 non-null	int64
6	Transport_Cost	5000 non-null	int64
7	Warehouse_Capacity	5000 non-null	int64
8	Warehouse_Utilization	5000 non-null	int64
9	On_Time_Delivery	5000 non-null	int64

dtypes: int64(10)

memory usage: 390.8 KB

None

MISSING VALUES

Order_ID	0
Region	0
Product	0
Demand	0
Inventory	0
Lead_Time	0
Transport_Cost	0
Warehouse_Capacity	0
Warehouse_Utilization	0
On_Time_Delivery	0

dtype: int64

STATISTICAL SUMMARY

	Order_ID	Region	Product	Demand	Inventory	\
count	5000.000000	5000.000000	5000.000000	5000.000000	5000.000000	

mean	2500.500000	1.503200	1.010000	273.702000	315.908200
std	1443.520003	1.115275	0.814145	130.886979	165.029457
min	1.000000	0.000000	0.000000	50.000000	30.000000
25%	1250.750000	1.000000	0.000000	150.000000	176.000000
50%	2500.500000	2.000000	1.000000	275.000000	316.000000
75%	3750.250000	2.000000	2.000000	388.000000	458.000000

```
#Data Cleaning
# Remove duplicates
df.drop_duplicates(inplace=True)
# Fill missing numeric values with median
numeric_cols = df.select_dtypes(include=np.number).columns
for col in numeric_cols:
    df[col].fillna(df[col].median(), inplace=True)
# Fill categorical missing values with mode
categorical_cols = df.select_dtypes(include='object').columns
for col in categorical_cols:
    df[col].fillna(df[col].mode()[0], inplace=True)
print("\nDATA CLEANED SUCCESSFULLY")
```

DATA CLEANED SUCCESSFULLY

```
#PHASE 1: DESCRIPTIVE ANALYSIS
#1. Regionwise Performance Analysis
region_perf = df.groupby('Region')['On_Time_Delivery'].mean().sort_values()
print("\nREGION PERFORMANCE")
print(region_perf)
plt.figure(figsize=(10,6))
region_perf.plot(kind='bar')
plt.title("Region Wise On-Time Delivery Performance")
plt.xlabel("Region")
plt.ylabel("Average On-Time Delivery %")
plt.xticks(rotation=45)
plt.show()
```

REGION PERFORMANCE

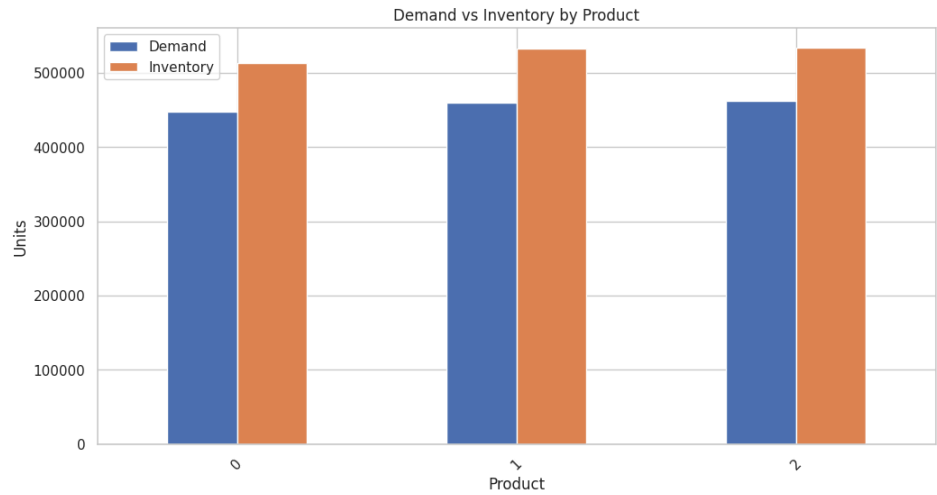
```
Region
3    0.490718
2    0.503507
1    0.508516
0    0.515663
Name: On_Time_Delivery, dtype: float64
```



```
# 2. PRODUCT-WISE DEMAND VS INVENTORY
product_analysis = df.groupby('Product')[['Demand', 'Inventory']].sum()
print("\nPRODUCT ANALYSIS")
print(product_analysis)
product_analysis.plot(kind='bar', figsize=(12,6))
plt.title("Demand vs Inventory by Product")
plt.ylabel("Units")
plt.xticks(rotation=45)
plt.show()
```

PRODUCT ANALYSIS

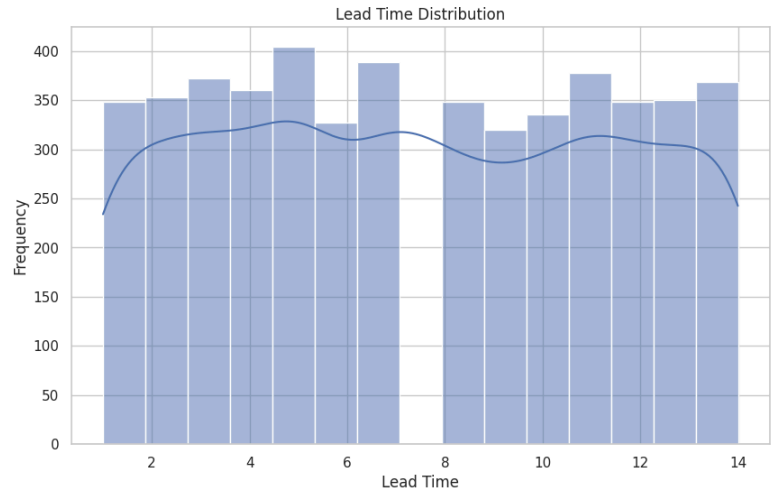
```
Product    Demand  Inventory
0         447478    513122
1         459446    532589
2         461590    533830
```



3. DELIVERY PERFORMANCE ANALYSIS

```
plt.figure(figsize=(10,6))
```

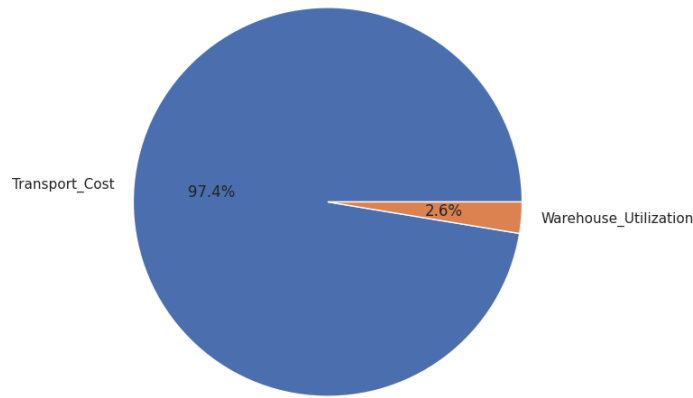
```
sns.histplot(df['Lead_Time'],
             bins=15,
             kde=True)
plt.title('Lead Time Distribution')
plt.xlabel('Lead Time')
plt.ylabel('Frequency')
plt.show()
```



```
#4 Cost Analysis
cost_analysis = df[['Transport_Cost',
                   'Warehouse_Utilization']].sum()
print("\nCOST ANALYSIS")
print(cost_analysis)
cost_analysis.plot(kind='pie',
                  autopct='%1.1f%%',
                  figsize=(7,7))
plt.title('Transport Cost vs Warehouse Utilization')
plt.ylabel('')
plt.show()
```

```
COST ANALYSIS
Transport_Cost    12987458
Warehouse_Utilization    346118
dtype: int64
```

Transport Cost vs Warehouse Utilization



```
#5 Identifying BottleNecks
bottlenecks = df[df['Lead_Time'] >
                 df['Lead_Time'].mean()]
print("\nBOTTLENECK RECORDS")
print(bottlenecks.head())
```

```
BOTTLENECK RECORDS
  Order_ID  Region  Product  Demand  Inventory  Lead_Time  Transport_Cost  \
2         3      2        0     220        441         12         3828
5         6      3        0     164        372         14         395
7         8      3        0     243        133         14        3629
8         9      1        1      52         74          8         360
9        10      2        0     498        505         12        1987

  Warehouse_Capacity  Warehouse_Utilization  On_Time_Delivery
2             1220             77              0
5             1819             80              0
7             1914             88              0
8             1094             92              0
9             1566             73              0
```

```
#6 REGION-WISE PERFORMANCE ANALYSIS
region_performance = df.groupby('Region')['On_Time_Delivery'].mean()
print(region_performance)
# INEFFICIENT REGION
inefficient_regions = region_performance[
    region_performance < region_performance.mean()
]
print("\nINEFFICIENT REGIONS")
print(inefficient_regions)
```

```
Region
0    0.515663
1    0.508516
2    0.503507
3    0.490718
```

Name: On_Time_Delivery, dtype: float64

INEFFICIENT REGIONS

Region
0.503507
0.490718

Name: On_Time_Delivery, dtype: float64

```
# HIGH COST DRIVERS
high_cost = df[
    df['Transport_Cost'] >
    df['Transport_Cost'].mean()
]
print("\nHIGH COST DRIVERS")
print(high_cost.head())
```

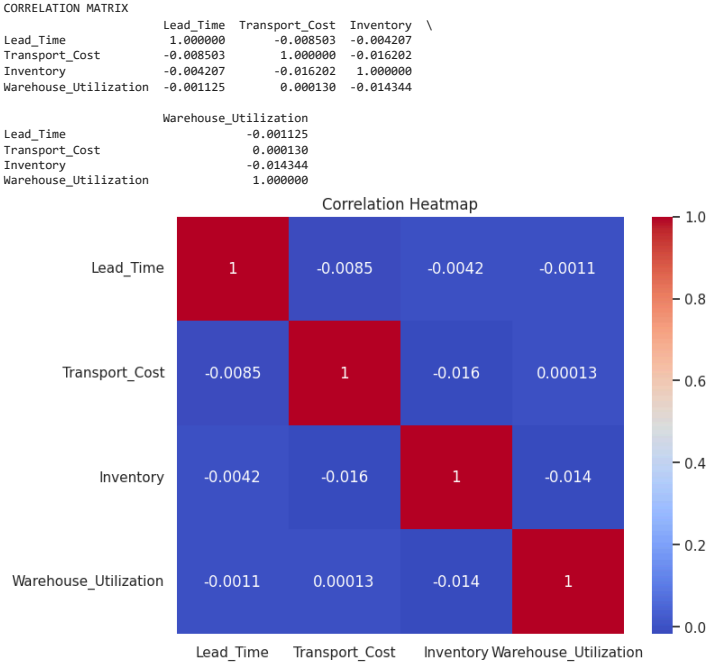
HIGH COST DRIVERS

Order_ID	Region	Product	Demand	Inventory	Lead_Time	Transport_Cost	\
1	2	1	0	110	518	7	3603
2	3	2	0	220	441	12	3828
7	8	3	0	243	133	14	3629
10	11	0	1	258	105	14	4532
12	13	1	2	231	303	13	4747

	Warehouse_Capacity	Warehouse_Utilization	On_Time_Delivery
1	983	49	1
2	1220	77	0
7	1914	88	0
10	968	70	0
12	598	46	0

```
# PHASE 2 : DIAGNOSTIC INSIGHTS
# WHY DELAYS HAPPEN?
correlation = df[['Lead_Time',
                  'Transport_Cost',
                  'Inventory',
                  'Warehouse_Utilization']].corr()

print("\nCORRELATION MATRIX")
print(correlation)
plt.figure(figsize=(8,6))
sns.heatmap(correlation,
            annot=True,
            cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



```
# STOCKOUT ANALYSIS
stockout = df[df['Demand'] > df['Inventory']]
print("\nSTOCKOUT CASES")
print(stockout.head())
print("\nTOTAL STOCKOUT CASES:", len(stockout))
```

STOCKOUT CASES

Order_ID	Region	Product	Demand	Inventory	Lead_Time	Transport_Cost	\
7	8	3	0	243	133	14	3629
10	11	0	1	258	105	14	4532
11	12	3	0	399	294	12	1044
16	17	0	2	458	66	6	4066
18	19	3	1	172	121	10	1955

	Warehouse_Capacity	Warehouse_Utilization	On_Time_Delivery
7	1914	88	0
10	968	70	0
11	519	52	0
16	1134	95	1
18	636	52	1

TOTAL STOCKOUT CASES: 2112

```
# MOST INEFFICIENT PRODUCT
product_efficiency = df.groupby('Product')['Lead_Time'].mean()
print("\nPRODUCT DELIVERY PERFORMANCE")
print(product_efficiency.sort_values(ascending=False))
```

PRODUCT DELIVERY PERFORMANCE

Product	
0	7.558824
1	7.435350
2	7.427467

Name: Lead_Time, dtype: float64

```
# PHASE 3 : PREDICTIVE ANALYTICS
# ENCODE CATEGORICAL VARIABLES
encoder = LabelEncoder()
for col in categorical_cols:
    df[col] = encoder.fit_transform(df[col])
# OPTION 1 : ON-TIME DELIVERY PREDICTION
X = df.drop('On_Time_Delivery', axis=1)
y = df['On_Time_Delivery']
# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
# Build Model
clf = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)
clf.fit(X_train, y_train)
# Prediction
y_pred = clf.predict(X_test)
# Evaluation
print("\nCLASSIFICATION ACCURACY")
print(accuracy_score(y_test, y_pred))
print("\nCONFUSION MATRIX")
print(confusion_matrix(y_test, y_pred))
print("\nCLASSIFICATION REPORT")
```

```
CLASSIFICATION ACCURACY
0.773

CONFUSION MATRIX
[[260 216]
 [ 11 513]]

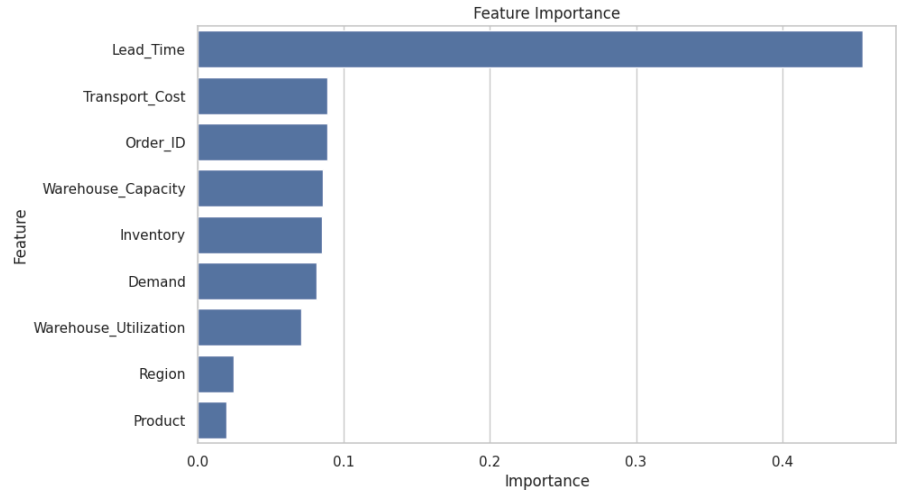
CLASSIFICATION REPORT
```

	precision	recall	f1-score	support
0	0.96	0.55	0.70	476
1	0.70	0.98	0.82	524
accuracy			0.77	1000
macro avg	0.83	0.76	0.76	1000
weighted avg	0.83	0.77	0.76	1000

```
# FEATURE IMPORTANCE
importance = pd.DataFrame({
    'Feature': X.columns,
    'Importance': clf.feature_importances_})
importance = importance.sort_values(by='Importance',ascending=False)
print("\nFEATURE IMPORTANCE")
print(importance)
plt.figure(figsize=(10,6))
sns.barplot(x='Importance',y='Feature',data=importance)
plt.title('Feature Importance')
plt.show()
# OPTION 2 : DEMAND PREDICTION
X_reg = df.drop('Demand', axis=1)
y_reg = df['Demand']
# Split Data
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(X_reg,
    y_reg,
    test_size=0.2,
    random_state=42)
# Build Model
reg = RandomForestRegressor(n_estimators=100,random_state=42)
reg.fit(X_train_reg, y_train_reg)
# Prediction
y_pred_reg = reg.predict(X_test_reg)
```

```
FEATURE IMPORTANCE
```

	Feature	Importance
5	Lead_Time	0.455253
6	Transport_Cost	0.088850
0	Order_ID	0.088587
7	Warehouse_Capacity	0.085724
4	Inventory	0.085281
3	Demand	0.081441
8	Warehouse_Utilization	0.070983
1	Region	0.024433
2	Product	0.019448



```
# DEMAND PREDICTION (REGRESSION)
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
# Features and Target
X_reg = df.drop('Demand', axis=1)
y_reg = df['Demand']
# Split Data
```

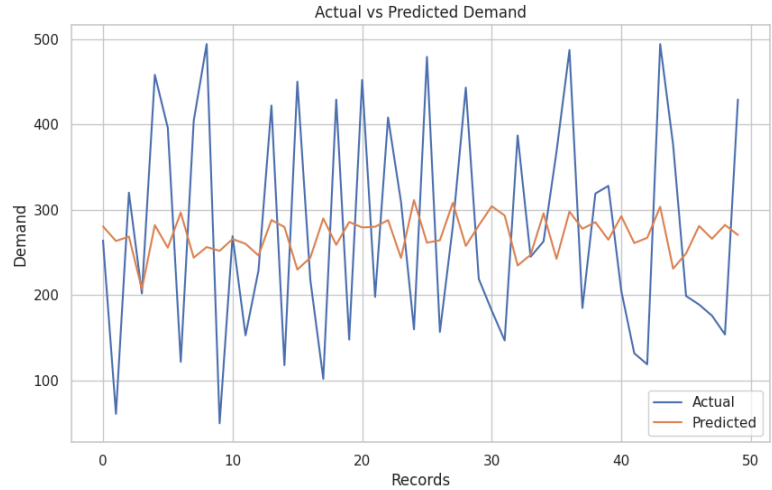
```
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg,
    y_reg,
    test_size=0.2,
    random_state=42)
# Build Model
reg = RandomForestRegressor(
    n_estimators=100,
    random_state=42)
# Train Model
reg.fit(X_train_reg, y_train_reg)
# Prediction
y_pred_reg = reg.predict(X_test_reg)
print("Regression Model Built Successfully")
# REGRESSION EVALUATION
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score)
print("\nREGRESSION RESULTS")
print("MAE:",
      mean_absolute_error(y_test_reg, y_pred_reg))
print("RMSE:",
      np.sqrt(mean_squared_error(y_test_reg, y_pred_reg)))
print("R2 SCORE:",
      r2_score(y_test_reg, y_pred_reg))
```

Regression Model Built Successfully

REGRESSION RESULTS
MAE: 116.09216
RMSE: 134.75929592054123
R2 SCORE: -0.05780221151003562

```
# ACTUAL VS PREDICTED DEMAND
comparison = pd.DataFrame({
    'Actual_Demand': y_test_reg,
    'Predicted_Demand': y_pred_reg})
print(comparison.head())
plt.figure(figsize=(10,6))
plt.plot(comparison['Actual_Demand'].values[:50],
         label='Actual')
plt.plot(comparison['Predicted_Demand'].values[:50],
         label='Predicted')
plt.title('Actual vs Predicted Demand')
plt.xlabel('Records')
plt.ylabel('Demand')
plt.legend()
plt.show()
```

	Actual_Demand	Predicted_Demand
1501	264	280.50
2586	61	263.45
2653	320	268.57
1055	202	207.15
705	458	282.02



```
#PHASE 4 Prescriptive Analytics
# INVENTORY OPTIMIZATION
df['Recommended_Inventory'] = df['Demand'] * 1.2
print("\nINVENTORY RECOMMENDATIONS")
print(df[['Product',
          'Demand',
          'Inventory',
          'Recommended_Inventory']].head())
```

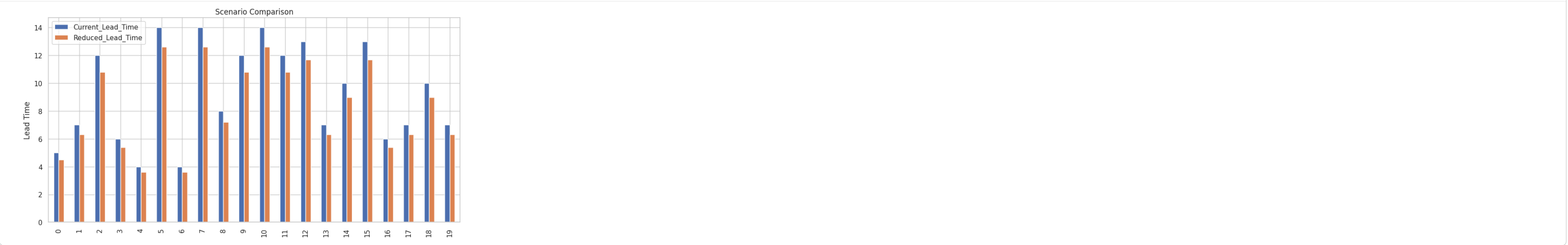
INVENTORY RECOMMENDATIONS				
	Product	Demand	Inventory	Recommended_Inventory
0	1	122	332	146.4
1	0	110	518	132.0
2	0	220	441	264.0
3	2	159	509	190.8
4	2	59	364	70.8

```
# REDUCE TRANSPORT COST
high_transport = df[
    df['Transport_Cost'] >
    df['Transport_Cost'].mean()
]
print("\nHIGH TRANSPORT COST CASES")
print(high_transport.head())
# WHAT-IF ANALYSIS
df['Reduced_Lead_Time'] = df['Lead_Time'] * 0.9
print("\nWHAT-IF ANALYSIS")
print(df[['Lead_Time',
          'Reduced_Lead_Time']].head())
```

HIGH TRANSPORT COST CASES							
	Order_ID	Region	Product	Demand	Inventory	Lead_Time	Transport_Cost \
1	2	1	0	110	518	7	3603
2	3	2	0	220	441	12	3828
7	8	3	0	243	133	14	3629
10	11	0	1	258	105	14	4532

12	13	1	2	231	303	13	4747
Warehouse_Capacity Warehouse_Utilization On_Time_Delivery \							
1	983	49	1				
2	1220	77	0				
7	1914	88	0				
10	968	70	0				
12	598	46	0				
Recommended_Inventory							
1	132.0						
2	264.0						
7	291.6						
10	309.6						
12	277.2						
WHAT-IF ANALYSIS							
Lead_Time		Reduced_Lead_Time					
0	5	4.5					
1	7	6.3					
2	12	10.8					
3	6	5.4					
4	4	3.6					

```
# SCENARIO COMPARISON CHART
scenario = pd.DataFrame({
    'Current_Lead_Time': df['Lead_Time'],
    'Reduced_Lead_Time': df['Reduced_Lead_Time']
})
scenario.head()
scenario[['Current_Lead_Time',
          'Reduced_Lead_Time']].head(20).plot(
    kind='bar',
    figsize=(12,6)
)
plt.title('Scenario Comparison')
plt.ylabel('Lead Time')
plt.show()
```

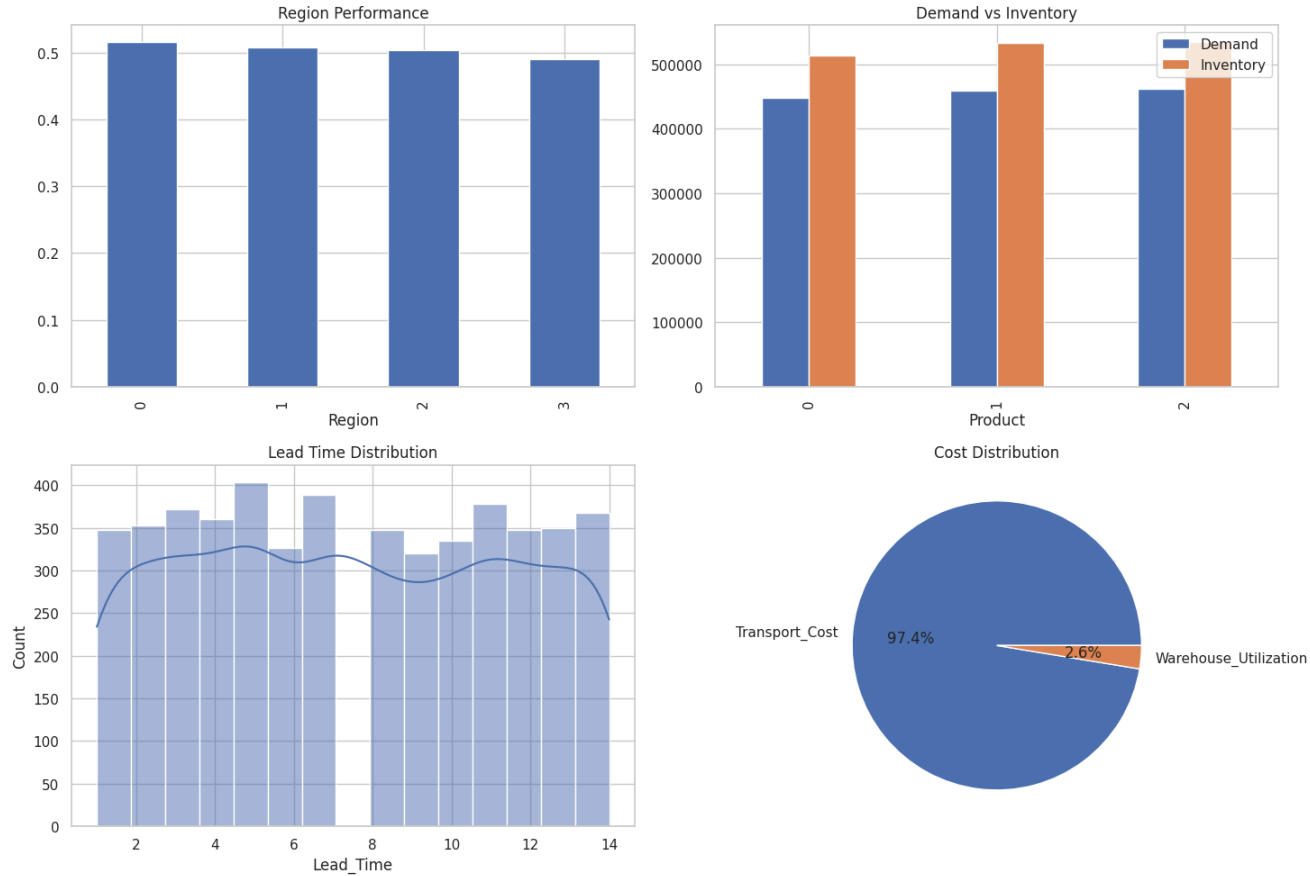


```
#Phase 5 Supply Chain Strategy
print("\nSUPPLY CHAIN STRATEGY")
strategies = [
    "Use AI/ML for demand forecasting",
    "Implement IoT for real-time shipment tracking",
    "Use automated warehouse systems",
    "Optimize inventory using predictive analytics",
    "Use dashboard monitoring for KPI tracking"
]
for strategy in strategies:
    print("-", strategy)
# PHASE 6 : DASHBOARD / VISUALIZATION
# KPI SUMMARY
total_demand = df['Demand'].sum()
total_inventory = df['Inventory'].sum()
average_lead_time = df['Lead_Time'].mean()
total_transport_cost = df['Transport_Cost'].sum()
print("\nKPI DASHBOARD")
print(f"TOTAL DEMAND: {total_demand}")
print(f"TOTAL INVENTORY: {total_inventory}")
print(f"AVERAGE LEAD TIME: {average_lead_time}")
print(f"TOTAL TRANSPORT COST: {total_transport_cost}")
# DASHBOARD CHARTS
fig, axes = plt.subplots(2,2, figsize=(15,10))
# Region Performance
region_performance.plot(
    kind='bar',
    ax=axes[0,0]
)
axes[0,0].set_title('Region Performance')
# Demand vs Inventory
product_analysis.plot(
    kind='bar',
    ax=axes[0,1]
)
axes[0,1].set_title('Demand vs Inventory')
# Lead Time Distribution
sns.histplot(df['Lead_Time'],
    bins=15,
    kde=True,
    ax=axes[1,0])
axes[1,0].set_title('Lead Time Distribution')
# Cost Analysis
cost_analysis.plot(
    kind='pie',
    autopct='%1.1f%%',
    ax=axes[1,1]
)
axes[1,1].set_ylabel('')
axes[1,1].set_title('Cost Distribution')
plt.tight_layout()
plt.show()
# EXPORT RESULTS
df.to_csv('cleaned_supply_chain_dataset.csv',
    index=False)
importance.to_csv('feature_importance.csv',
```

```
index=False)
print("\nPROJECT COMPLETED SUCCESSFULLY")

SUPPLY CHAIN STRATEGY
- Use AI/ML for demand forecasting
- Implement IoT for real-time shipment tracking
- Use automated warehouse systems
- Optimize inventory using predictive analytics
- Use dashboard monitoring for KPI tracking

KPI DASHBOARD
TOTAL DEMAND: 1368514
TOTAL INVENTORY: 1579541
AVERAGE LEAD TIME: 7.473
TOTAL TRANSPORT COST: 12987458
```



PROJECT COMPLETED SUCCESSFULLY

Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.